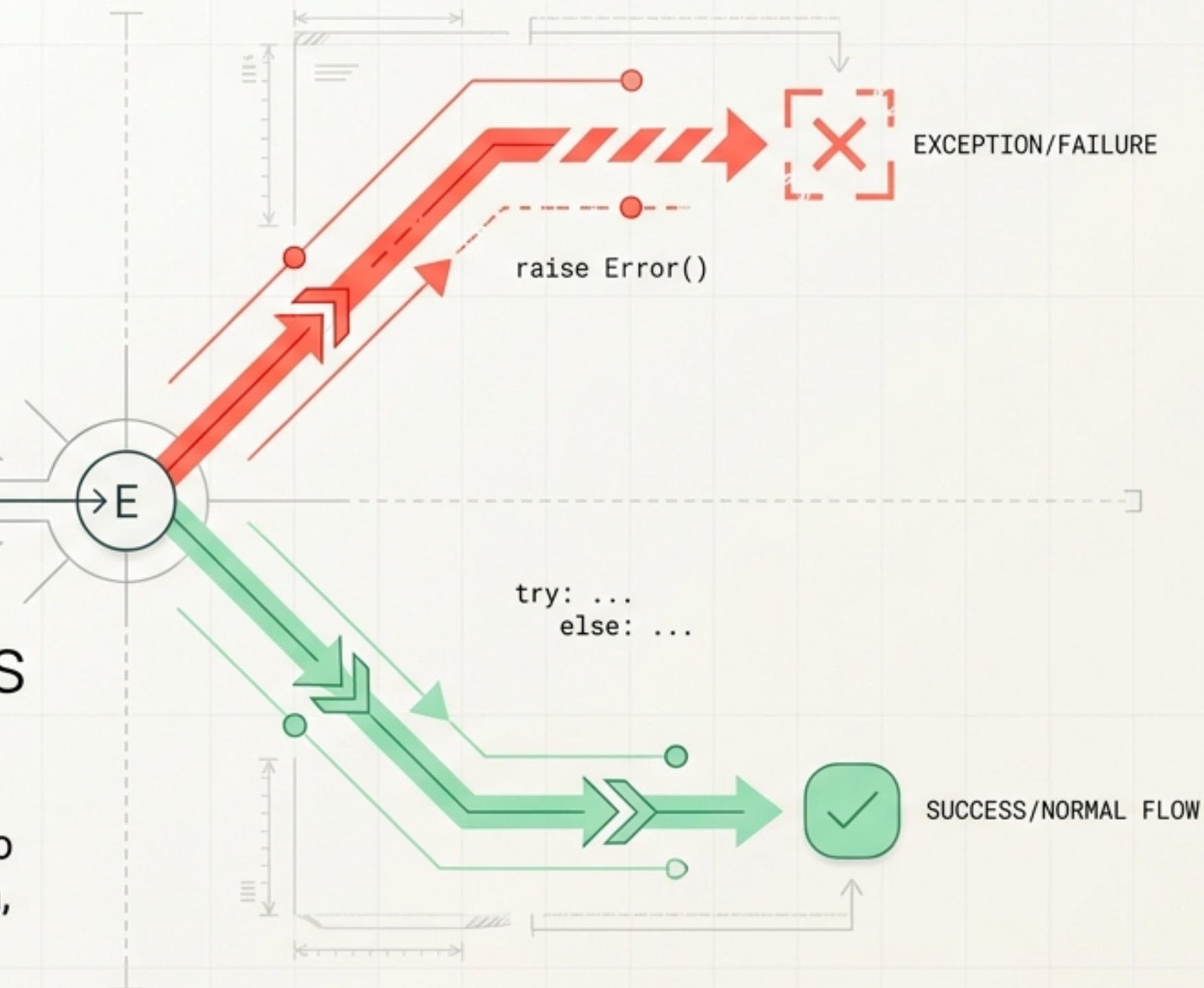
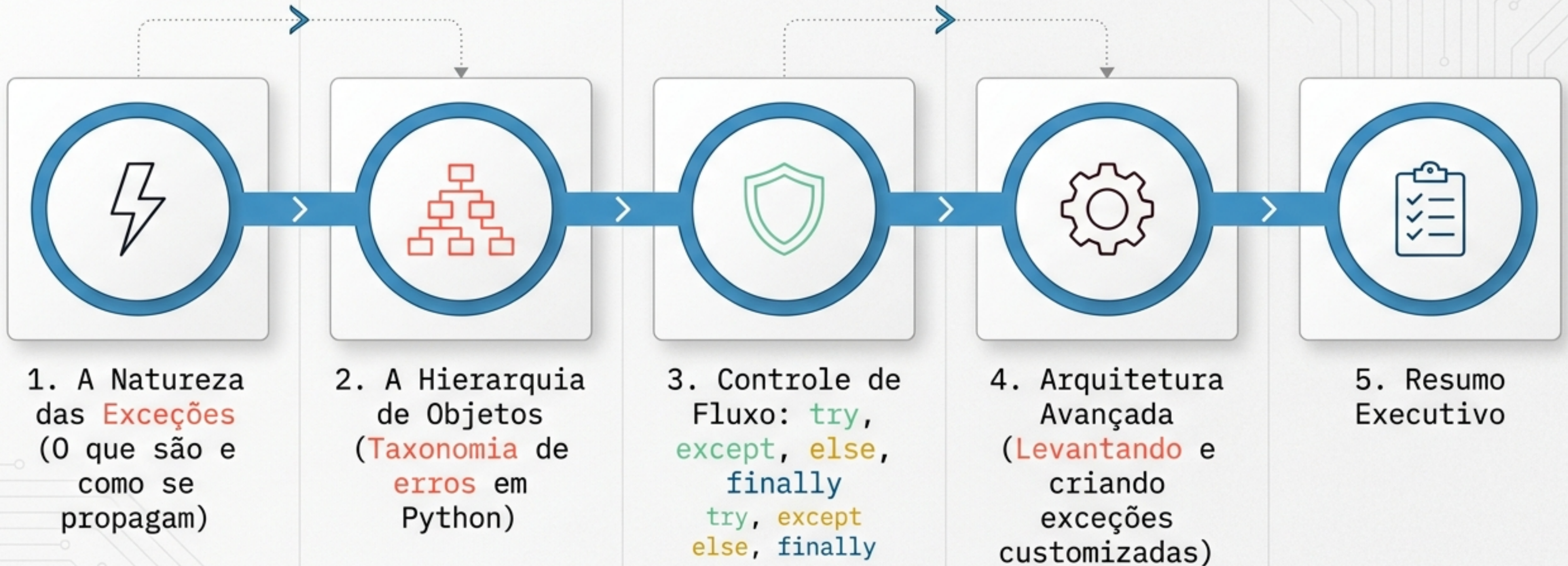


Tratamento de Exceções em Python

De falhas inesperadas à arquitetura de código resiliente. Uma referência visual para captura, propagação e customização de erros.



Sumário da Apresentação



Anatomia de uma Falha Inesperada

Quando um erro não é tratado, o programa é encerrado imediatamente. O Python gera um objeto de Erro e imprime um Traceback (Rastreamento de Pilha).

Seta 1: Indica o arquivo exato e o número da linha onde o erro ocorreu.

Run:  properties x

 /Users/Shared/workspaces/pycharm/venv/python /Users/Shared/workspaces/pycharm/pythonintro/properties/pro

Traceback (most recent call last):

File `"/Users/Shared/workspaces/pycharm/pythonintro/properties/properties.py"`, line 32, in `<module>`

`print(person)`

File `"/Users/Shared/workspaces/pycharm/pythonintro/properties/properties.py"`, line 28, in `__str__`

`return 'Person[' + str(self._name) + '] is ' + self._age`

`TypeError: Can't convert 'int' object to str implicitly`

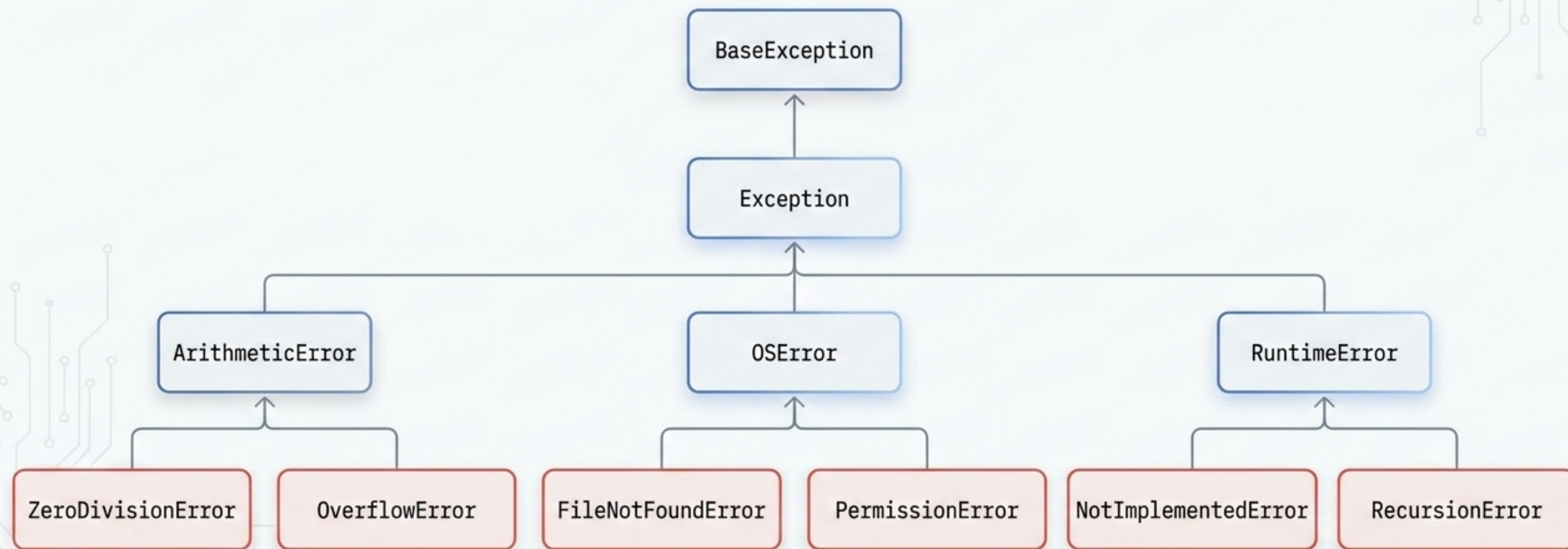
Process finished with exit code 1

Seta 2: A classe específica do erro gerado.

Seta 3: Mensagem clara descrevendo o que causou o problema.

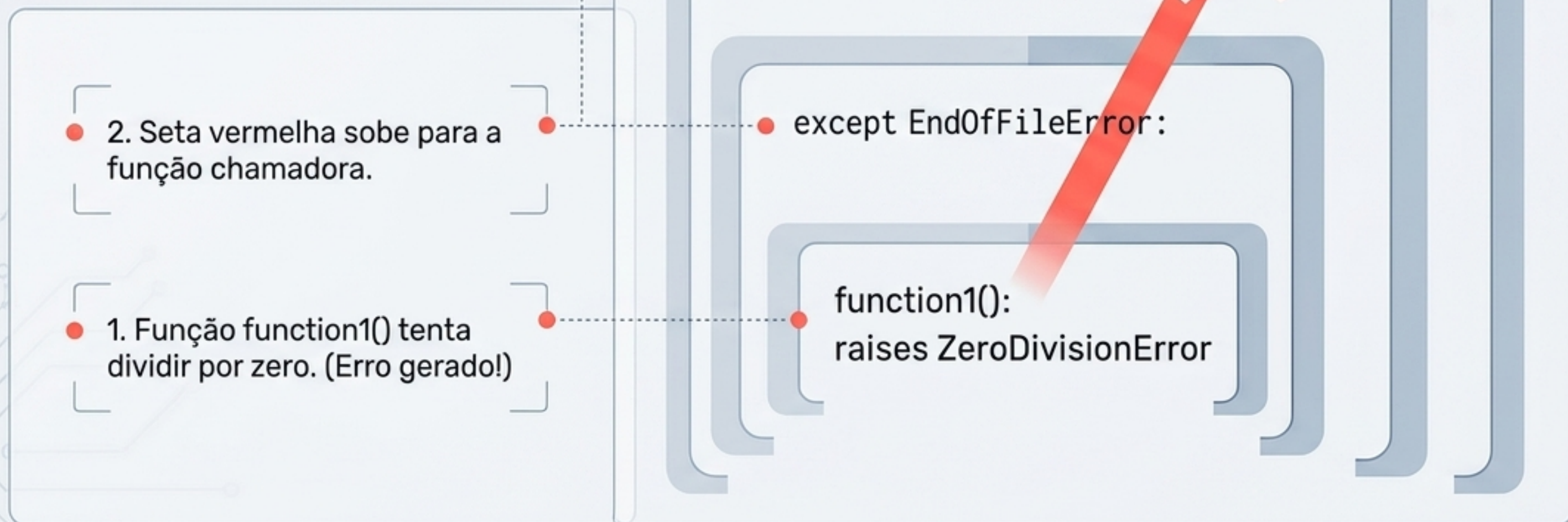
A Taxonomia das Exceções em Python

Em Python, tudo é um objeto. Todas as exceções herdam de uma classe base comum. Compreender esta árvore é vital, pois capturar uma classe pai também captura todos os seus filhos.



O Efeito Bolha na Pilha de Execução

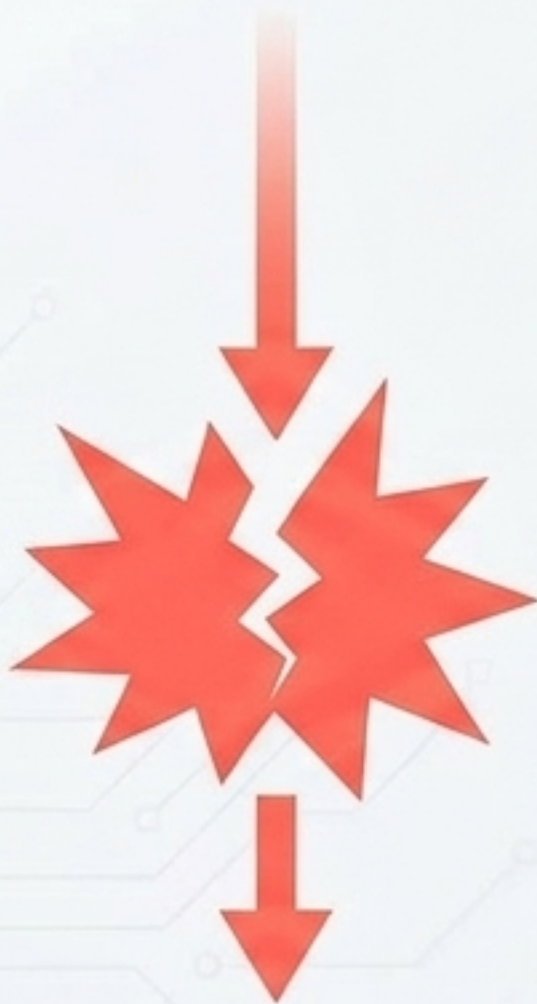
Quando uma exceção é gerada (raised), ela interrompe o fluxo normal e procura um tratador (handler). Se não encontrar, ela sobe para a função que a chamou, até encontrar um bloco de captura ou derrubar o programa.



Interceptando a Falha com try-except

O bloco **try** isola o código arriscado. Se ocorrer um erro compatível, a execução salta imediatamente para o bloco **except**, ignorando o restante do código dentro do **try**.

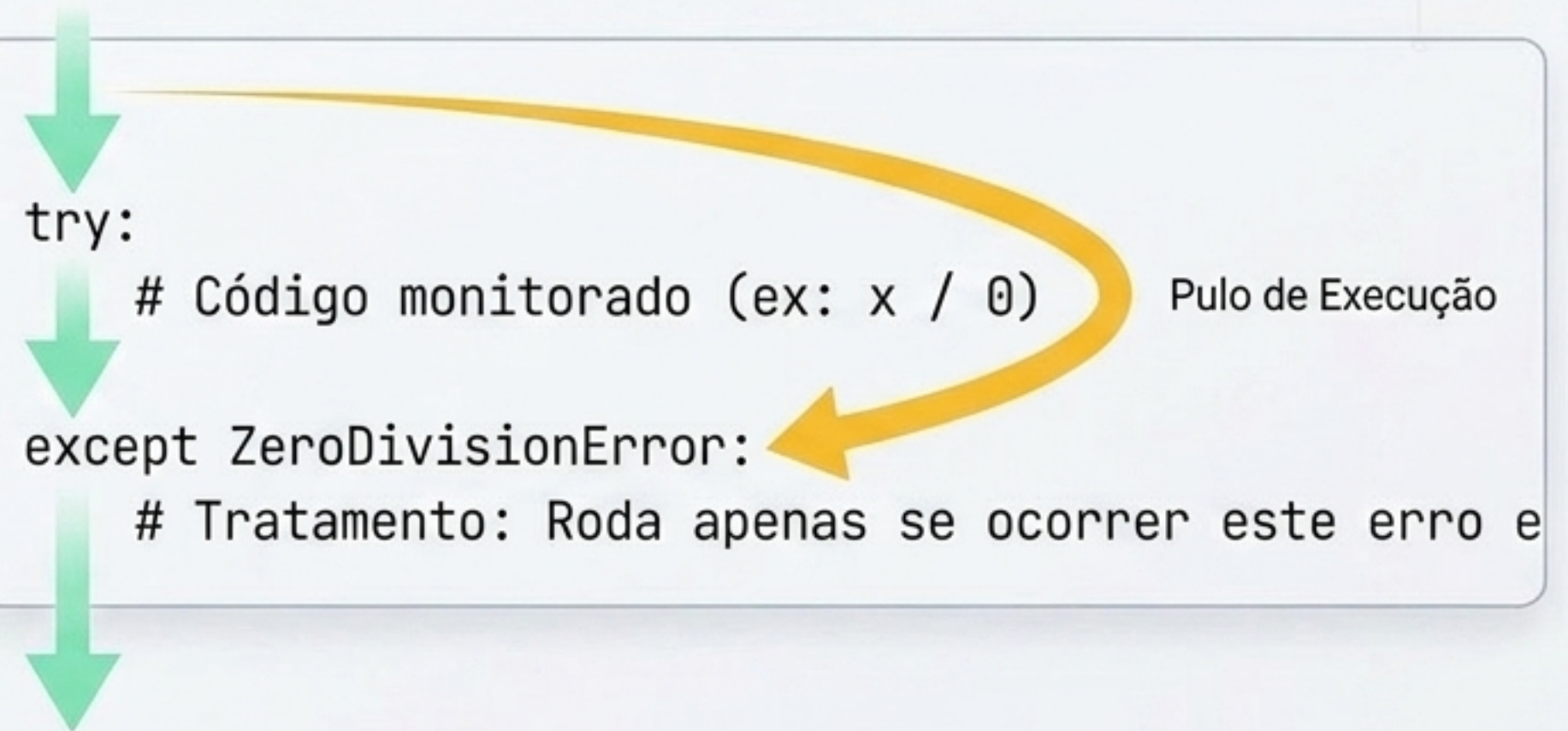
Sem Proteção



Com try-except

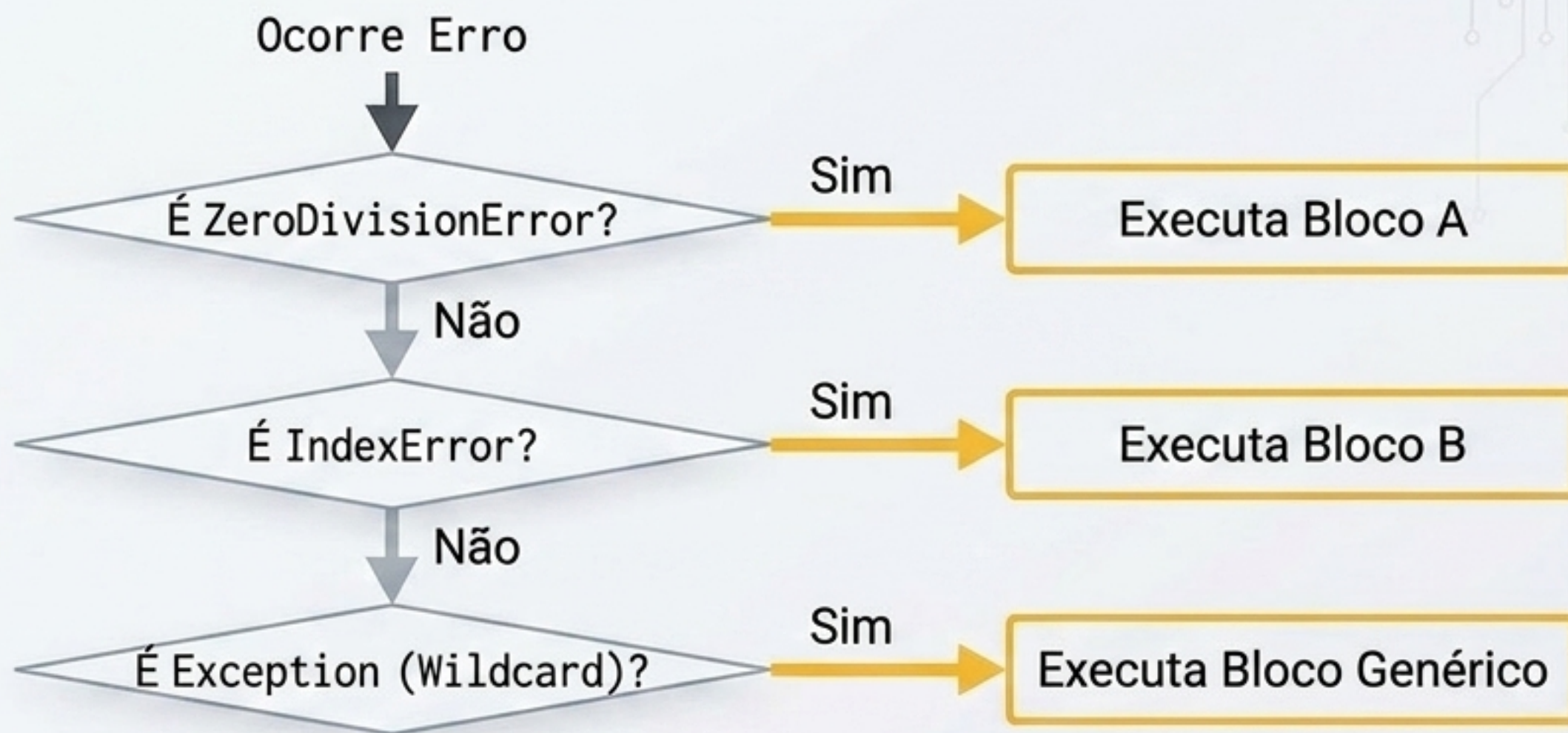
```
try:
    # Código monitorado (ex: x / 0)
except ZeroDivisionError:
    # Tratamento: Roda apenas se ocorrer este erro e
```

Pulo de Execução



Roteamento de Múltiplas Exceções

É possível tratar diferentes erros de maneiras diferentes. O Python avalia os blocos **except** de cima para baixo. A regra de ouro é: classes filhas devem vir antes das classes pai.



Aviso: Colocar 'Exception' no topo **bloquearia a execução de todos os outros blocos abaixo dele.**

Inspecionando o Objeto da Exceção

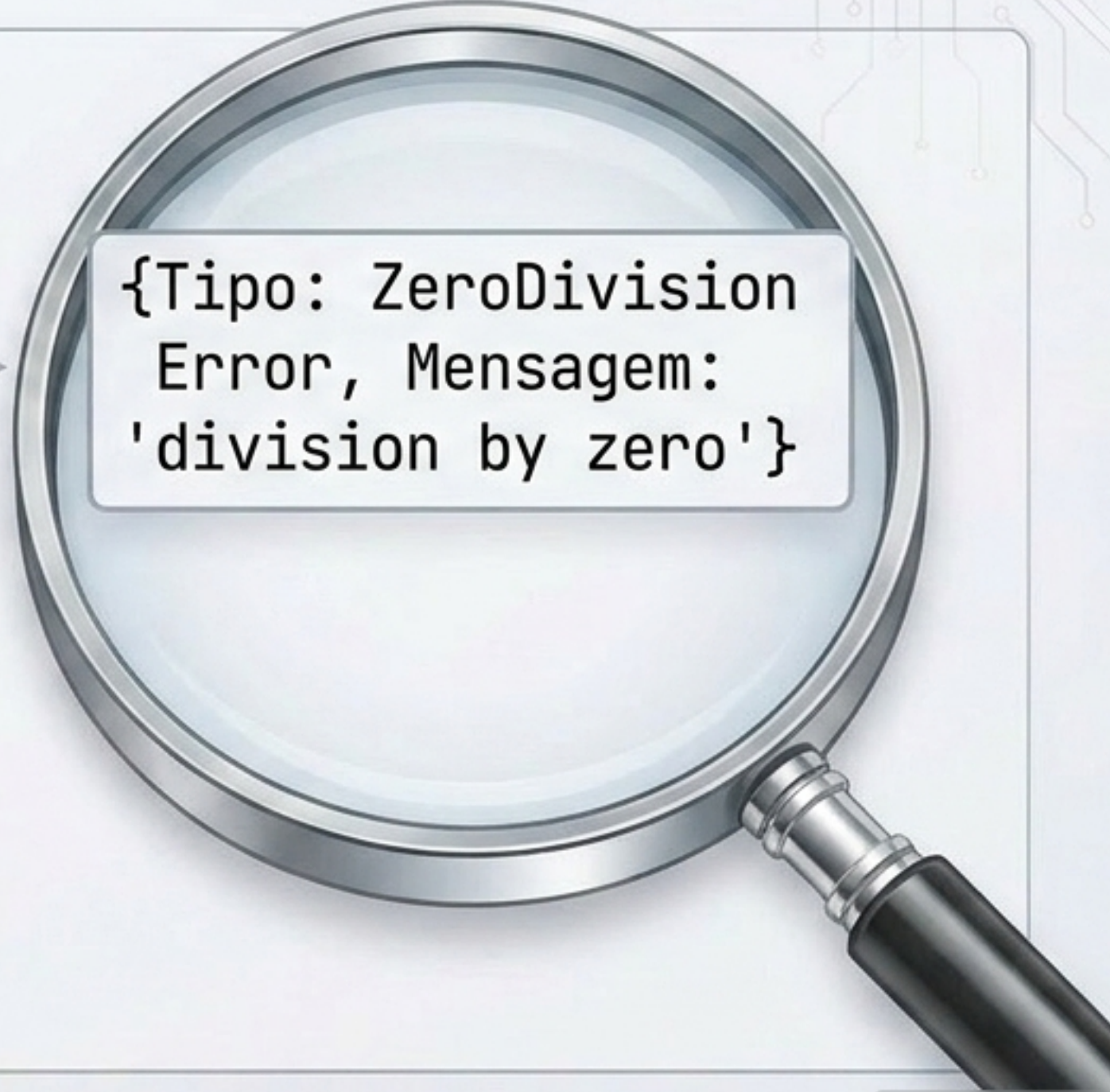
Capturar o erro é o primeiro passo; entendê-lo é o segundo. A palavra-chave **'as'** vincula o objeto da exceção gerada a uma variável local, permitindo inspecionar sua mensagem e propriedades.

```
try:
```

```
    runcalc(6)
```

```
except ZeroDivisionError as exp:
```

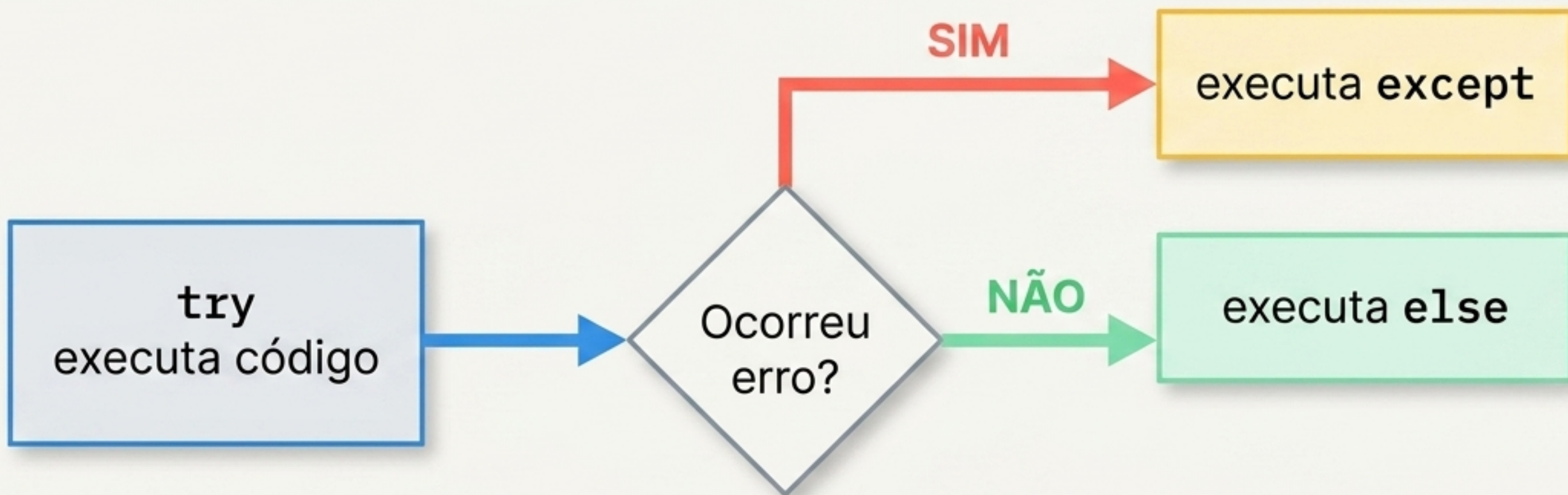
```
    print('Detalhe:', exp)
```



```
{Tipo: ZeroDivision  
Error, Mensagem:  
'division by zero'}
```

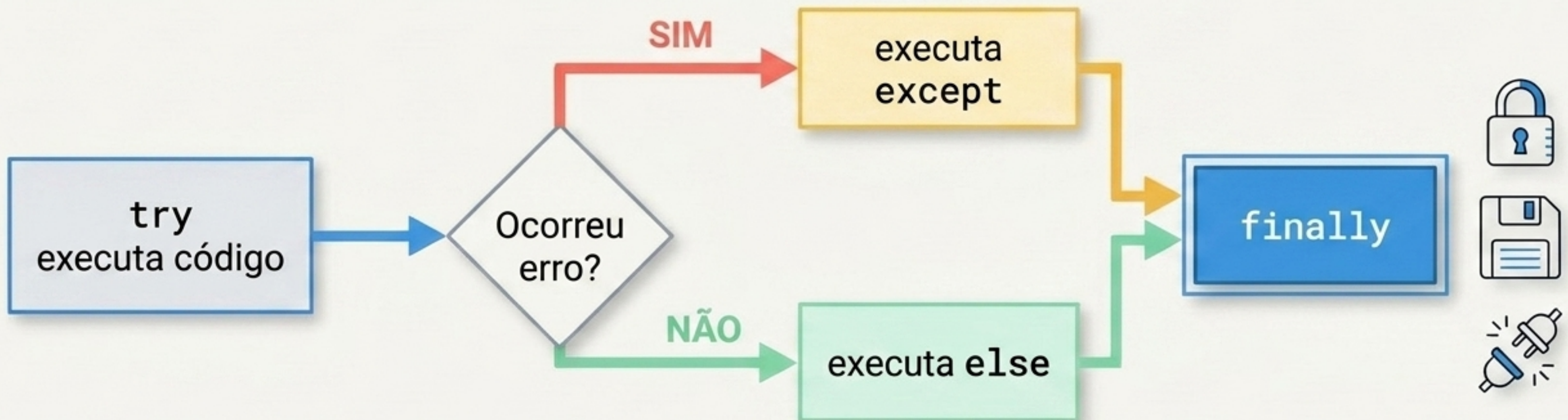

A Execução Limpa com a Cláusula **else**

O bloco **else** só é executado se nenhuma exceção for lançada no bloco **try**. Ele previne que você coloque muito código não-arriscado dentro da área monitorada.



0 Encerramento Garantido com **finally**

A cláusula **finally** roda sempre, independentemente de sucesso ou falha no código monitorado. É o local arquitetônico correto para fechamento de recursos (como fechar arquivos ou derrubar conexões de banco de dados).



Matriz Estrutural de Tratamento de Erros

Cláusula	Quando Executa?	Propósito Principal	Obrigatório?
try	Primeiro (Sempre)	Monitorar código de risco	Sim 
except	Se houver falha	Capturar e tratar o erro	Sim (ou finally) 
else	Se houver sucesso	Rodar código dependente do sucesso	Não
finally	No encerramento (Sempre)	Limpar e liberar recursos	Não

Gerando Erros Intencionais com **raise**

Como desenvolvedor, você não precisa esperar o sistema quebrar sozinho. O comando **raise** permite instanciar e lançar erros proativamente quando uma condição de negócio é violada.

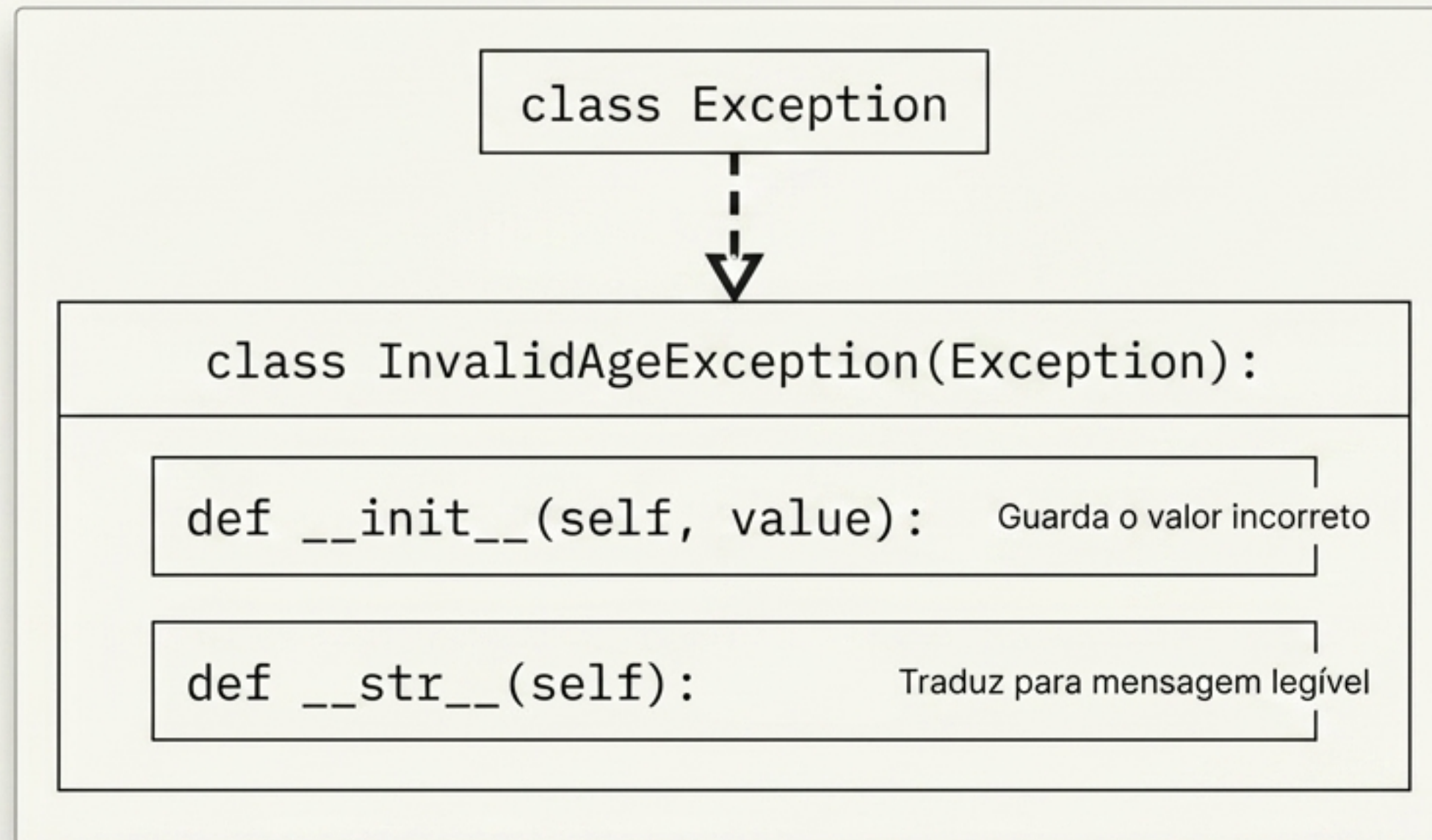
Data Validation

```
if not isinstance(value, int):  
    raise ValueError('Idade inválida!')
```



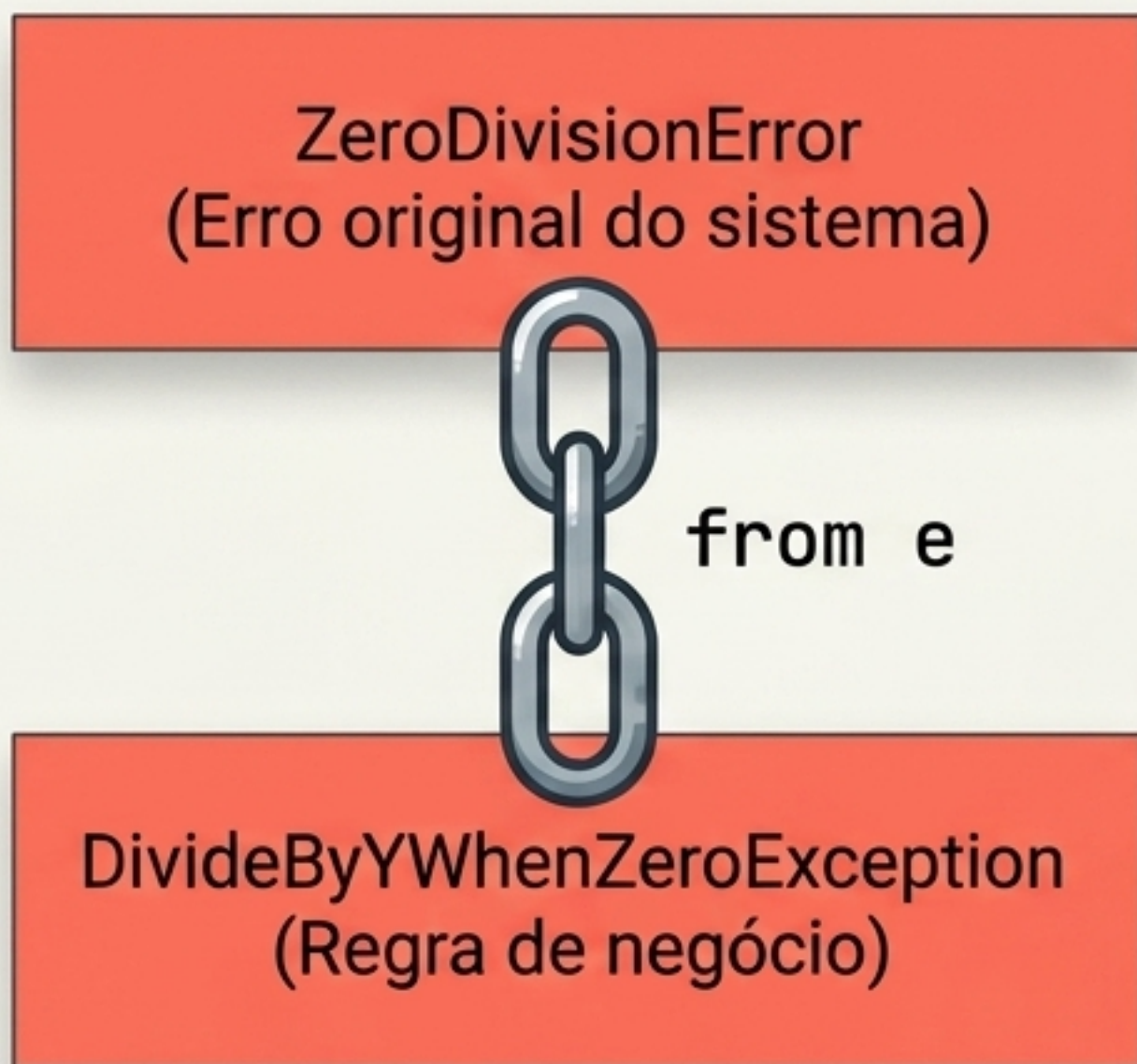
Arquitetura de Exceções Customizadas

Para obter mais controle granular em aplicações complexas, crie suas próprias exceções. Basta herdar da classe `Exception` e definir o comportamento do objeto, provendo semântica clara ao seu domínio.



Encadeamento de Exceções para Rastreabilidade

Ao empacotar um erro de sistema em um erro de domínio customizado, use `raise ... from e`. Isso vincula o erro original ao novo, preservando toda a trilha de auditoria para debugging.



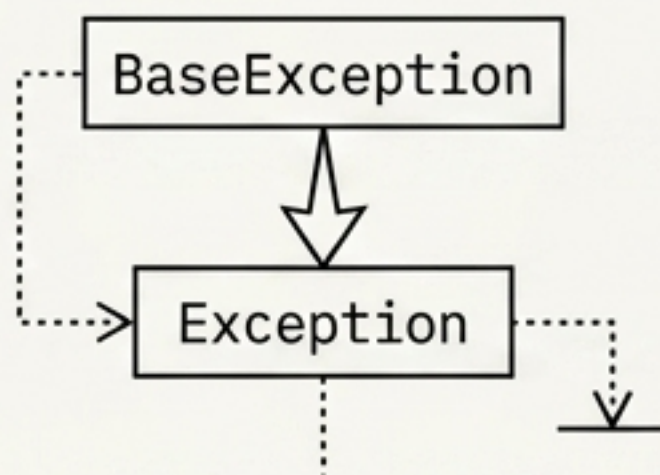
```
Traceback (most recent call last):  
  File "/zeroDivisionError.py", line 473, in <module>  
    raise ZeroDivisionError + exception:  
    raise *exception tk = 0
```

The above exception was the direct cause of the following exception:

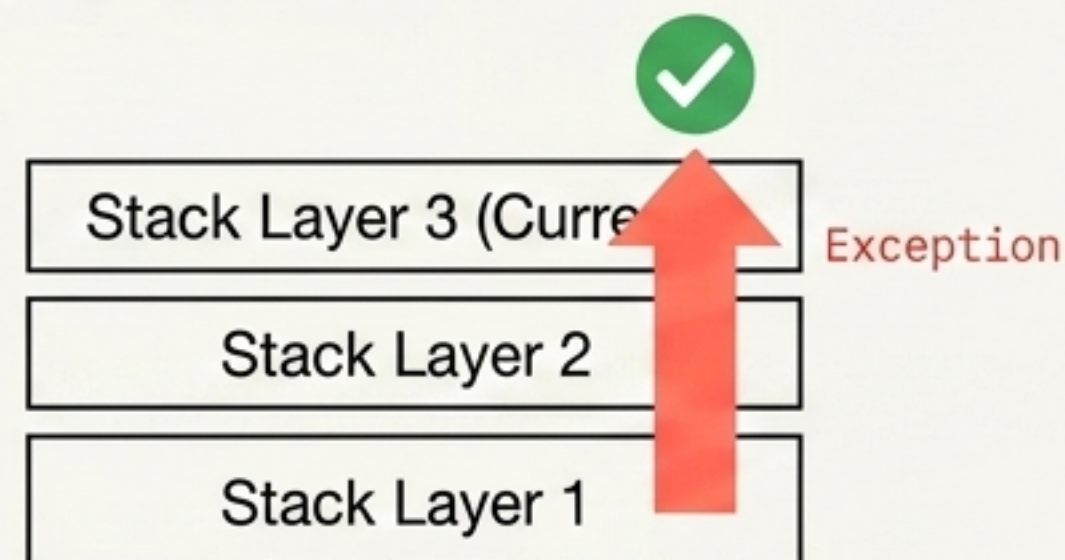
```
Traceback (most recent call last):  
  File "/core/svimin.py", line 41 in <module>  
    DivideByYWhenZeroException is negócio :  
    DivideByYWhenZeroException
```


Resumo Executivo

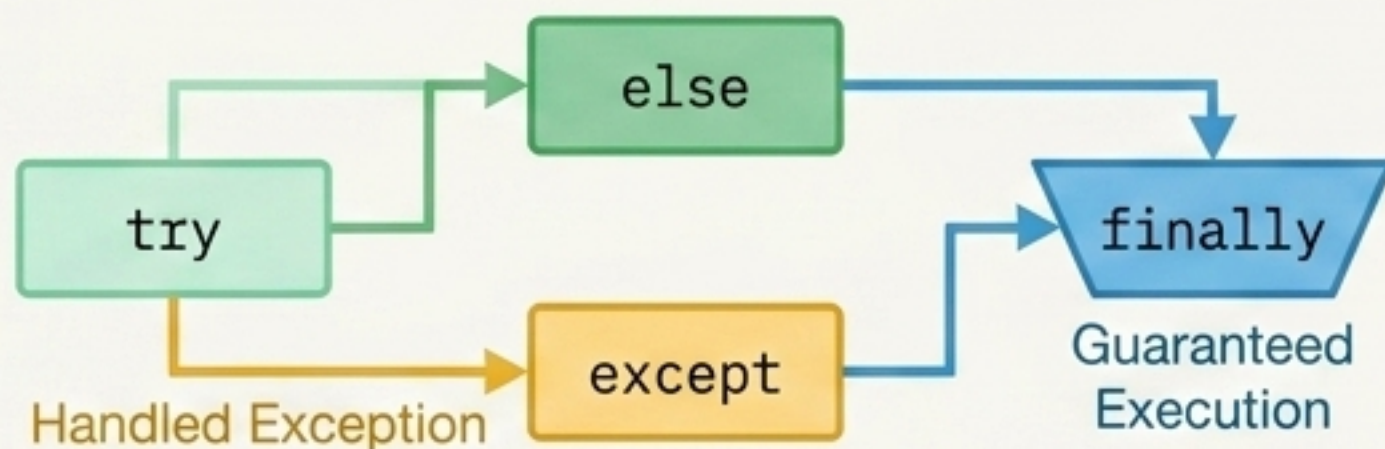
1. Tudo é Objeto



2. O Efeito Bolha



3. Fluxo de Captura



4. Controle Total

Uso do comando `raise` para disparar erros intencionais e herança de classes para construir Exceções Customizadas baseadas em regras de domínio.

